TECHNICAL SPECIFICATION · DRAFT V0.2

Online Judge Design Spec

A system that accepts untrusted source code from strangers, executes it safely, judges it deterministically, and survives the sixty seconds when everyone submits at once — designed end to end before a line of code is written.

AUTHOR Meet Modi	STATUS Revised after review
DATE 5 Sep 2026	SCOPE Level 1 → Level 4
COMPANION Online Judge LLD	

VERDICT VOCABULARY AC WA TLE MLE RE CE OLE IE

CONTENTS

01 Problem Statement
02 Overview
03 Scope
04 Users & Journeys
05 Roadmap L1 → L4
06 Requirements
07 Capacity Model
08 High Level Design
09 Challenges
10 Interface Summary
11 Security Posture
12 Decisions
13 Deployment
14 Testing
15 Observability
16 Risks
17 Milestones

01 Problem Statement

A coding challenge is a competitive event in which participants solve algorithmic problems within a fixed timeframe. Solutions are submitted as source code, evaluated automatically against concealed test cases, and scored on the results.

An **Online Judge** is the platform that hosts these challenges. It must accept untrusted source code from thousands of strangers, execute it safely, evaluate it deterministically, and return a trustworthy verdict — while staying up during the exact minute everyone submits at once.

Two things make this hard, and they are the whole project:

1. **You are running arbitrary code from the internet on your own machines.** Every submission is a potential fork bomb, an `rm -rf /`, a crypto miner, or an attempt to read the test case files sitting next to it.
2. **Load is not uniform.** An online judge is idle for twenty-three hours and then takes its entire day's traffic in the final ninety seconds of a contest.

02 Overview

> ## Online Judge = Online Compiler + Processing Logic

	INPUT	OUTPUT	SEMANTICS
Online Compiler	code + *optional* custom stdin	raw stdout / stderr / exit code	Synchronous. No judgement. `POST /run`
Online Judge	code + `problem_id`	a **verdict**	Asynchronous. Same executor, N hidden test cases, plus comparison. `POST /submit`

The judge is the compiler, plus a comparison loop, plus a scoring model. Build the compiler correctly and the judge is a thin layer on top; build it badly and no amount of judge logic saves you.

What makes this build different. Most online judge clones stop at “submit → verdict”. This one adds an AI Mentor whose answers are **grounded in real execution, not an LLM's guess** — the Dry-Run Simulator traces your code for real, and the Complexity Auditor measures your runtime at N = 10^2, 10^3, 10^4, 10^5 and fits the curve. The execution plane built for judging becomes the mentor's evidence source. See §8.6.

03 Scope

In scope by end of Level 4: auth, problem catalogue, code editor, online compiler, sandboxed judging, contests, live leaderboard, profiles, problem ingestion, AI Mentor (six modes), deployment, load testing, observability.

Explicitly out of scope: plagiarism detection (MOSS), payments, mobile apps, collaborative editing, video editorials, and writing our own compiler — we shell out to `g++`, `javac` and `python3`.

04 Users & Core Journeys

PERSONA	CORE JOURNEY
Learner	Browse problems → open one → write code → *Run* with custom input → *Submit* → read verdict → get unstuck via the AI Mentor
Contestant	Join contest → solve under time pressure → watch live standings → review after
Setter / Admin	Author or ingest a problem → attach or generate test cases → set limits → publish

05 Roadmap: Level 1 → Level 4

The spine of the project. Each version is independently shippable and demoable — nothing from V3 starts before V2 is deployed.

VER	LEVEL	THEME	WHAT LANDS
V0	L1–L2	Skeleton	Express + Mongo + React. JWT auth. Problem CRUD. Monaco editor. `POST /run` executing on the host via `child_process`. Runs on `localhost` only — **V0 is never deployed**, and the host executor is unreachable outside local development by four independent guards (§11.5).
V1	L3	Product	Docker sandbox replaces host execution. Test cases and real verdicts. `POST /submit` goes async through a Redis/BullMQ work queue with a separate worker. Contests, history, profile. One `docker-compose` stack on a single EC2 box.
V2	L4a	Distributed	Kafka as the event backbone. Leaderboard becomes an event-sourced Redis projection with WebSocket push. Test cases move to S3. Worker fleet autoscales on queue depth. Gateway, rate limiting, OpenTelemetry. Ingestion service.
V3	L4b	Intelligence	AI Mentor: six modes, execution-grounded, with a solution-leak guard, response cache and per-user token budget.
V4	L4c	Hardening	Load tests, chaos drills, golden-verdict regression suite, blue/green deploys, runbooks, cost model.

06 Non-Functional Requirements

REQUIREMENT	TARGET
Verdict latency at contest peak	`p50 < 5 s · p95 < 30 s · p99 < 60 s`
`POST /submit` API response	`< 200 ms — it only enqueues, it never judges`
`POST /run` round trip	`< 8 s (5 s execution cap + overhead)`
Availability	99.5% overall. **Submission ingestion must survive a total judge-fleet outage** — the queue absorbs, verdicts drain later.
Verdict correctness	100% deterministic and reproducible for the same (code, test case, limits)
Isolation	No submission reaches the network, the host filesystem, another submission, or the test case store
Fairness	Timing measured as **CPU time on a dedicated core**, never wall-clock under contention

07 Capacity Model

Sizing is done for one realistic contest, not a hand-wave.

```
Design target        : 10,000 registered users, 2,000 concurrent in a 2-hour contest
Submissions          : 2,000 users x 8 submissions          = 16,000 submissions
Average rate         : 16,000 / 7,200 s                     ~ 2.2 submissions/s
Peak (last 10 min)   : ~25% of volume in ~10% of the window ~ 5.5/s sustained, ~20/s spike

Cost of one judgement
  compile (C++, cached on repeat)                           ~ 0.6 s
  20 test cases x ~0.15 s typical                           ~ 3.0 s
  container start / teardown                                ~ 0.4 s
  ------------------------------------------------------------------------
  p50 ~ 4 s     worst case (all 20 cases hit a 1 s limit)   ~ 25 s

Concurrency needed — Little's Law, L = lambda x W
  steady : 5.5/s x 4 s    ~  22 judge slots
  spike  : 20/s  x 4 s    ~  80 judge slots

Slot footprint: 1 vCPU + 256 MB. A 16-vCPU node hosts ~14 slots.
  => 2 worker nodes steady, 6-8 at peak. Autoscale on queue depth, not CPU.
```

The queue is what makes this work. We do not size the fleet for the spike. We size for the steady rate, absorb the spike in the queue, and hold the p95 by draining. That is the entire answer to the thundering-herd problem.

Validating these assumptions

Every number above is a planning hypothesis derived from guesses about user behaviour. **None of it is treated as settled.** The computed fleet size sets the initial autoscaling bounds — it is not a fixed capacity, and it is not a commitment.

ASSUMPTION	VALUE USED	REPLACED BY	SENSITIVITY
Submissions per user per contest	`8`	Beta contest telemetry	Linear on fleet size
Median judge duration	`4 s`	k6 run over the golden submission mix	**Linear, and the dominant term**
Test cases per problem	`20`	Catalogue statistics at ~100 problems	Linear on judge duration
Container start overhead	`400 ms`	Measured on the target instance type	Decides whether the warm pool is worth it (§12)
Peak concentration	`25% in 10%`	Contest telemetry	Affects queue depth and p95, not steady fleet size
Slots per node	`cores − 2`	Load test with timing-variance checks	If timings degrade under load, fewer slots per node

The V4 load test (§14) exists to replace this table's left column with measurements. **Its deliverable is a revised §7, not a pass/fail.** Until that revision exists, treat the worker count here as an order-of-magnitude estimate that happens to have arithmetic behind it.

08 High Level Design

8.1 Architecture

The system splits into three planes with different trust levels, different scaling curves, and different failure blast radii.

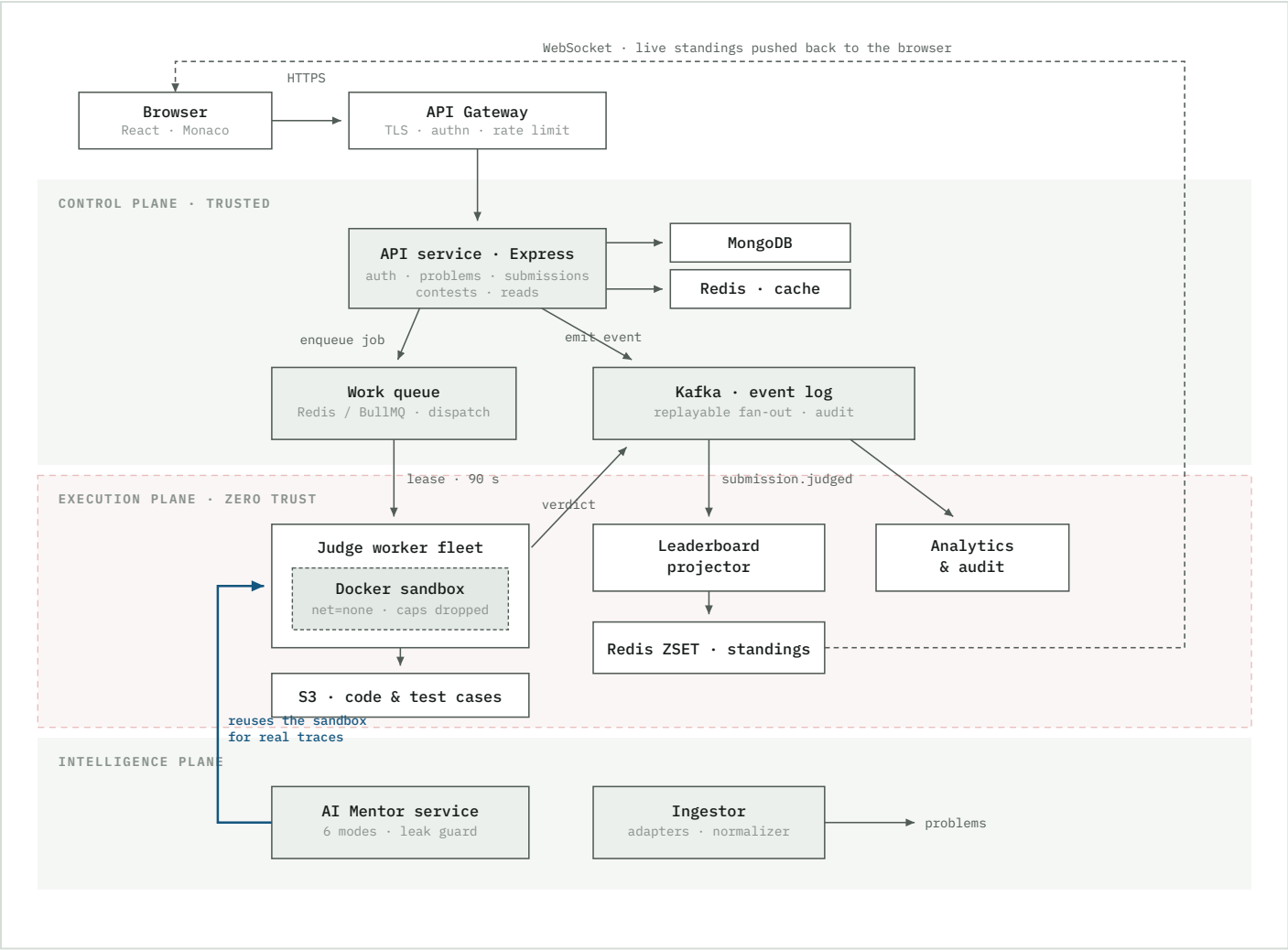


FIG 1 Untrusted code executes only inside the dashed zero-trust band; the control plane never compiles or runs anything. The accented edge is the project's differentiator — the AI Mentor borrows the same sandbox to produce evidence rather than guesses.

8.2 Why three planes

PLANE	TRUST	SCALES WITH	FAILURE IMPACT
Control	Trusted	Users online	Site down — but no data loss
Execution	**Zero trust** — hostile code runs here	Submissions/sec	Verdicts delayed, queue keeps filling. Site stays up.
Intelligence	Trusted code, untrusted output	AI requests + token budget	Mentor unavailable. Judging unaffected.

Keeping these separate is the single most important structural decision. **The API server must never compile or execute code.** If it does, one fork bomb takes down the website.

The second structural rule is that dependencies point *inward*, never outward. The control plane depends on nothing in the intelligence plane; the execution plane depends on neither. Concretely: **the judging path can run with the entire intelligence plane deleted**, and that is verified by a chaos drill rather than assumed (§8.6).

8.3 Flow: `POST /run` — the Online Compiler

Synchronous, no verdict, custom stdin, tighter limits.

1. API validates `{ language, code, stdin? }` and enforces a per-user token bucket in Redis.
2. Job pushed to the **high-priority** queue — interactive traffic must never queue behind a contest backlog.
3. Worker writes `<uuid>.<ext>` to a scratch dir, compiles, runs sandboxed with a 5 s wall cap and 256 MB.
4. Returns `{ stdout, stderr, exitCode, timeMs, memoryKb }`, output truncated at 64 KB.
5. Scratch dir deleted in a `finally` block — always, including on timeout.

8.4 Flow: `POST /submit` — the Judge

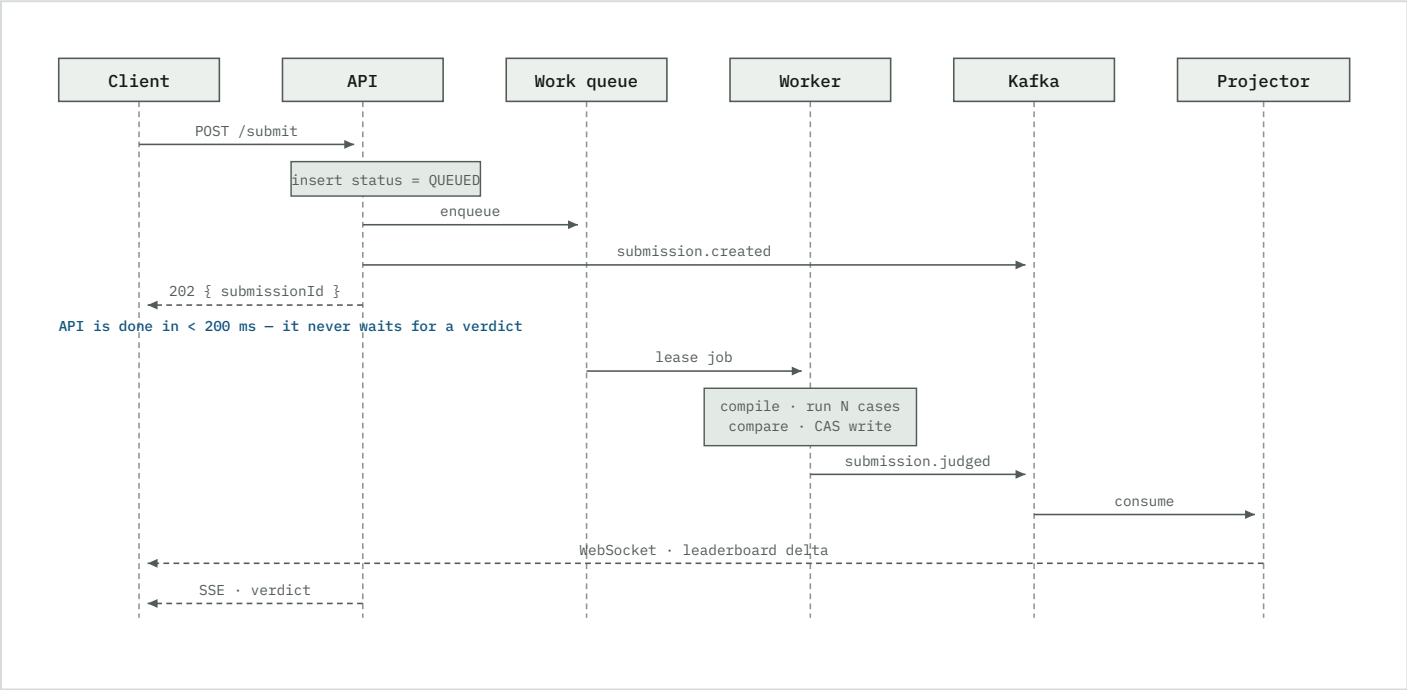


FIG 2 The request path and the verdict path are different paths. The API acknowledges and exits; the verdict arrives later over a channel the client is already holding open.

The verdict write is a **compare-and-set on status** (`QUEUED|COMPILING|RUNNING → final`), so a redelivered job can never overwrite a verdict that already landed. That is how we get idempotency without distributed transactions.

8.5 Leaderboard

Read-heavy (everyone refreshes) and write-light (only on judged submissions) — so it is a **materialized projection**, never a live query against the submissions collection.

- The projector consumes `submission.judged` and maintains a Redis **sorted set** per contest.
- ICPC-style score packed into one double so a single `ZREVRANGE` is the whole read: `score = solved × 10⁹ − penaltySeconds`. Safe — doubles hold $2^{53} \approx 9\times10^{15}$ exactly.
- Clients get the top 50 over WebSocket. Deltas are pushed, not polled.

- **Degradation:** if Redis is cold or lost, the projector replays the Kafka topic from the contest-start offset and rebuilds. Nothing is lost, because the event log — not Redis — is the source of truth.

8.6 AI Mentor — the X-factor

Six modes, each mapped to a specific place a learner gets stuck. The design principle: **an answer the system can verify beats an answer the model asserts.**

01	**Intuition Engine**	*"I have no clue."* Breaks down the statement, classifies the pattern (sliding window, two pointers, grid search), names the edge cases — without dumping code.
02	**Brainstorming Partner**	*"I have a clue and started."* A sounding board for partial logic. Finds structural gaps, validates edge-case reasoning, nudges toward code without writing it.
03	**Dry-Run Simulator** `REAL EXECUTION`	*"Written an approach, not sure it's correct."* Not a model pretending to be a CPU. The code is instrumented and run in the sandbox on a small input; the **actual variable trace** is fed to the model, which narrates it and points at the exact line where state diverges.
04	**Complexity Auditor** `REAL MEASUREMENT`	*"Solved, but TLE."* Generates inputs at N = 10^2, 10^3, 10^4, 10^5, measures runtime, fits the curve to an **empirical complexity**, then explains the hotspot and the optimisation that removes it.
05	**Conceptual Translator**	*"I don't understand this data structure."* Drops code entirely. Metaphors, diagrams and invariants — why a segment tree query is O(log N), how a trie routes nodes.
06	**Foundations Architect**	*"New to this data structure."* Primitives first — operations, insertion and deletion, memory layout — then the three canonical problems, pulled from our own catalogue by tag.

Modes 3 and 4 are the differentiator, and they are only possible because we already built a safe execution plane. Reuse it.

Three guardrails, non-negotiable:

1. **Leak guard.** The mentor must never emit a working solution. Enforced in layers: mode-scoped system prompts; an output filter that rejects a response containing a complete compilable solution (full signature + control flow + return, above a length threshold); and per-problem editorial-similarity checks.
2. **Contest mode.** The mentor is disabled during a rated contest, or its usage is recorded on the submission. Decided per contest, never left implicit.
3. **Context is assembled server-side.** The client sends `{ problemId, mode, editorBuffer }`. It does not get to declare what it is allowed to see — the service pulls the statement, the user's own recent verdicts, and sample-only test data.

Cost control: responses cached on `(problemId, mode, sha256(code))`; a per-user daily token budget in Redis; a cheap model for modes 1, 5 and 6 and a stronger one for 3 and 4; SSE streaming so time-to-first-token stays low.

8.7 Problem Ingestion

An offline pipeline, never on the hot path. A pluggable `SourceAdapter` per site → normalizer → `problems` collection, retaining `source`, `sourceUrl` and attribution.

> **FLAGGING THIS HONESTLY**
>
> Scraping statements from LeetCode or Codeforces raises copyright and terms-of-service issues. Default behaviour is **metadata plus link only** (title, tags, difficulty, URL) for third-party sources; full statements are stored only for our own or permissively-licensed problems. Codeforces has an official API — prefer it. This keeps the pipeline useful and the project publishable.

Scraped problems arrive without test cases. So the setter path is **`generator.py` + `brute.cpp`** — generate M inputs, run the brute-force reference to produce expected outputs, store the set with a `version` hash. This is how test data is actually authored in competitive programming, and it makes the platform self-sufficient.

8.8 The mentor is optional by construction

The judge is the product; the mentor is an enhancement. If the intelligence plane is what takes the platform down during a contest, the differentiator has become the liability. So the independence is structural rather than a matter of care:

- **No inbound dependency.** `api` and `judge-worker` never import the mentor. A lint rule and a CI dependency-graph check fail the build if that changes.
- **No synchronous call from the hot path.** The mentor is reachable only at `POST /api/ai/*`. Nothing internal calls it, so an LLM provider timeout can never occupy an API worker or delay a submission.
- **Read-only data access.** The mentor holds a database credential with no write grant. It cannot corrupt a submission, a verdict or a standing even through a bug.
- **Separate deployment and a kill switch.** Its own process, health check and autoscaler; `AI_ENABLED=false` disables it instantly without touching anything else.

Verified, not asserted. A V4 chaos drill scales the mentor to zero replicas mid-contest. Pass condition: every submission still judged, standings still updating, and the only visible change is the mentor panel reporting itself unavailable. Contract detail in **LLD §11.7**.

---

09 Challenges & Solutions

	CHALLENGE	SOLUTION
1	**Thundering herd** — 2,000 people submit in the last 60 seconds	Never judge inline. `POST /submit` writes `QUEUED` and enqueues in under 200 ms. Workers drain at a fixed rate. Size for the steady rate, absorb the spike in the queue, hold p95 by draining. Autoscale on queue depth.
2	**Malicious code** — fork bombs, `rm -rf`, miners, reading the test case files	Docker sandbox: `--network=none`, read-only rootfs, tmpfs workdir, non-root user, all capabilities dropped, pid and memory and CPU caps, seccomp profile, host-side kill timer. Guarantees in §11.2; exact configuration in LLD §6.4.
3	**Verdict tampering**	Verdicts are written only by workers, only via CAS, only over the internal network. No client-supplied verdict field is ever trusted. Every transition is emitted to Kafka as an immutable audit log.
4	**Leaderboard read storm**	Event-sourced Redis ZSET projection plus WebSocket push. Zero submission-table queries on the read path.
5	**False `TLE` from noisy neighbours**	Measure **CPU time from the cgroup**, not wall-clock. Pin each judge slot to a dedicated core. Run a submission's test cases **sequentially** — parallelising them inside one submission creates contention and non-reproducible timings.
6	**The AI hands out the answer**	Three-layer leak guard (§8.6) plus a contest-mode kill switch.
7	**AI cost runs away**	Cache on (problem, mode, code hash), per-user token budgets, model routing by mode.
8	**Large test data**	Test cases live in S3 — not in Kafka messages, not in Mongo documents. Workers cache them on local disk keyed by `testcaseSetVersion`. Messages carry references, never payloads.
9	**Poison-pill submissions** that crash the worker every time	Attempt counter on the job. After three failures the submission is marked `IE`, routed to a DLQ and alerted on — it never blocks the queue.
10	**Container escape** — a kernel CVE defeats the sandbox entirely	Treated as inevitable rather than impossible. Containment is designed so an escaped attacker lands on a credential-less, network-isolated, 24-hour-lifetime node with no route to the control plane, the database or the internet. Full posture and blast-radius statement in §11.3.
11	**Language gotchas**	Java requires `public class Main` in `Main.java`. The filename is forced, and a mismatch surfaces as a clear `CE` rather than a confusing failure. Language registry in LLD §6.5.

10 Interface Summary

The contracts between components, at the level a reviewer needs to judge the architecture. Field-by-field schemas, indexes, middleware ordering, comparator semantics and every algorithm live in the companion **Online Judge LLD**, which is authoritative wherever the two documents differ.

10.1 Data model

COLLECTION	HOLDS	NOTE
`users`	Account, role, rating, solved count	—
`problems`	Statement, limits, checker type, test case set version	The version hash is what invalidates worker caches
`testcases`	Input/output references, sample flag	Data itself lives in S3 from V2
`submissions`	One row per submit: status, verdict detail, timing	Append-mostly and the hot collection — never rewritten in bulk
`contests`	Window, problem set, scoring mode, mentor flag	—
`refresh_tokens`	Rotation lineage for auth	Stored hashed; reuse revokes the family
`outbox`	Verdict events awaiting publish	Guarantees a verdict is never lost between the database and Kafka
`judge_runs`	Per-test-case execution record	14-day TTL; feeds debugging and the Complexity Auditor

Submission source and test case data are object-store references, never embedded documents. Full schemas and the query each index serves: **LLD §3**.

10.2 API surface

METHOD	ROUTE	AUTH	BEHAVIOUR
POST	`/api/auth/register`	—	Create user, bcrypt hash
POST	`/api/auth/login`	—	Access JWT (15 min) + refresh token in an httpOnly cookie
POST	`/api/auth/logout`	user	Revoke refresh token
GET	`/api/problems`	—	Paginated list; filters `tags`, `difficulty`, `q`
GET	`/api/problems/:slug`	—	Statement, limits, and **sample test cases only**
POST	`/api/problems`	setter	Create problem
POST	`/api/run`	user	`{language, code, stdin?}` → stdout, stderr, exit code, timing. Rate limited 10/min.
POST	`/api/submissions`	user	`{problemId, language, code, contestId?}` → **202** `{submissionId, status:'QUEUED'}`
GET	`/api/submissions/:id`	owner	Current status and verdict detail
GET	`/api/submissions/:id/stream`	owner	SSE — pushes each status transition until terminal
GET	`/api/users/:username`	—	Profile and solved stats

METHOD	ROUTE	AUTH	BEHAVIOUR
GET	/api/contests/:id/leaderboard	—	Top-N straight from the Redis ZSET
WS	/ws/contests/:id	user	Live standings deltas
POST	/api/ai/ask	user	`{problemId, mode, editorBuffer}` → SSE stream. Budget-checked, leak-guarded.

Per-route behaviour — middleware order, token rotation, rate limit buckets, pagination, the submit sequence: **LLD §4**.

10.3 Events

TOPIC	KEY	PARTS	CONSUMERS	WHY THIS KEY
submission.created	submissionId	12	analytics	Even spread, no ordering requirement
submission.judged	contestId ?? userId	12	projector, analytics, notifications	**All events for one contest land in one partition**, so the projector sees them in order with no cross-partition coordination. Tradeoff: a hot partition per contest — fine at 20 events/s.
problem.ingested	problemId	3	search indexer	—
ai.interaction.logged	userId	3	budget accounting	—
submission.judged.dlq	submissionId	1	on-call alerting	—

Envelope schema, producer and consumer settings, offset handling: **LLD §2 and §8**.

10.4 Verdict lifecycle

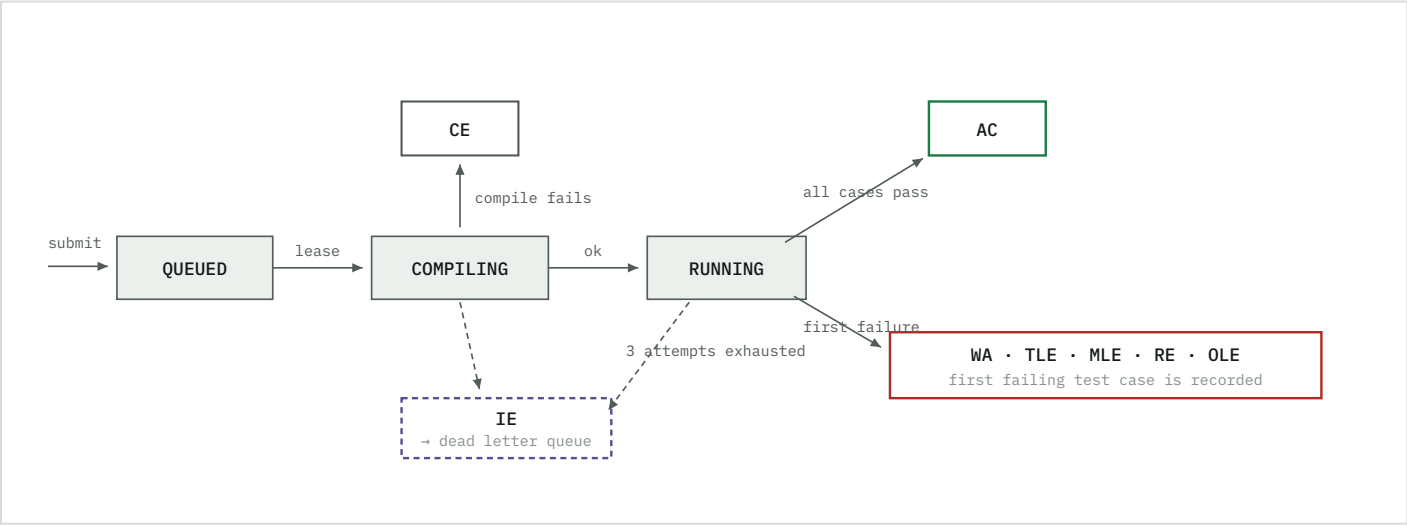


FIG 3 Every terminal state is written once. IE is dashed because it is reachable from anywhere and means *our* failure, not the user's — it is the state that keeps a poison pill from blocking the queue.

Every terminal status is written exactly once, by a compare-and-set against the pending states. A worker whose write matches zero documents knows another worker already finished and exits quietly — that is how a redelivered job is made harmless without distributed transactions. The write and its Kafka event share one transaction; see **LLD §7.6**.

11 Security Posture

This platform's core function is to execute code written by anonymous strangers. Security is therefore not a section appended to the design — it is the constraint the architecture was shaped around. This section states what is defended, by what, and what an attacker gets when a layer fails.

11.1 The trust boundary

There is exactly one: the edge of the execution plane (Fig 1, dashed band). Untrusted code runs inside it and nowhere else. The API service, the projector and the mentor never compile or execute a submission, so no amount of hostile input to them results in code execution.

11.2 What the sandbox guarantees

Stated as properties rather than flags. The exact container configuration is **LLD §6.4**.

PROPERTY	MECHANISM
No network access of any kind	The container joins no network — not a restricted one, none
Bounded memory	Hard limit with swap disabled, so exceeding it is `MLE` and not a host OOM
Bounded, measurable CPU	One pinned core; CPU time accounted from the cgroup, not the wall clock
No fork bombs	Process-count limit inside the container
No privilege escalation	All capabilities dropped, `no-new-privileges`, non-root uid, seccomp allowlist
Immutable filesystem	Read-only rootfs; the single writable mount is `noexec`
No disk exhaustion	File-size ulimit plus a 64 KB output cap → `OLE`
No access to answers	Test cases arrive on stdin; nothing is ever mounted into the container

11.3 Infrastructure containment — assuming the sandbox fails

A container boundary is one kernel bug away from not being a boundary. The design assumption here is that it *will* eventually fail — a kernel CVE, a runtime regression, a flag we got wrong. So the question this section answers is not “can they escape” but **“what do they get if they do”**.

The answer is: a bare, credential-less node with no route to anything that matters.

CONTROL	WHAT IT DENIES AN ATTACKER WHO IS ALREADY ROOT ON THE NODE
Dedicated, tainted node pool	No control-plane component, no API pod, no projector and no other service shares that kernel. Judge nodes run judge workers and nothing else.
No cloud credentials on the node	Instance metadata is blocked at the network layer (IMDSv2, hop limit 1, plus a firewall rule on `169.254.169.254`). The node's instance profile grants nothing. The worker's S3 access is a short-lived scoped token held in process memory, never written to disk.
No cluster secrets	`automountServiceAccountToken: false`, no image-pull secrets, no kubeconfig, no database password anywhere on the host.
Egress denied by default	The judge subnet has no NAT gateway and no internet route. Outbound is allowed only to the S3 VPC endpoint, Redis and Kafka, on specific ports. **An escaped process cannot phone home, exfiltrate, or pull a second stage.**
Ingress denied	No inbound rules except the health check from the load balancer subnet.
Nothing durable on the node	Ephemeral root volume, tmpfs scratch, and a 24-hour maximum node lifetime. An implant does not survive the day.
Least-privilege data reach	A VPC endpoint policy scopes S3 to the code and test case prefixes, read-only for test cases. Reaching S3 does not mean reaching the bucket.
Kernel hygiene	Auto-patched node images, a forced monthly AMI roll, and user-namespace remapping so container root maps to an unprivileged host uid.
Upgrade path	If we ever host public untrusted contests, the runtime moves to gVisor or Firecracker microVMs — a per-submission kernel boundary rather than a shared one. Costed in §16, not adopted speculatively.

Blast radius. The worst realistic outcome is one compromised, credential-less, network-isolated node that is destroyed within twenty-four hours. It holds no user data beyond the test cases for problems being judged at that moment, and it cannot reach the control plane, the database, the object store beyond two prefixes, or the internet.

11.4 Detection and response

Containment without detection means an attacker gets twenty-four hours undisturbed. Judge nodes run a syscall monitor (Falco, or auditd rules) alerting on:

- `execve` of anything outside the runner's allowlisted binaries
- writes outside the tmpfs scratch mount
- any outbound connection attempt — there should never be one
- `setuid`, kernel module loads, or writes to `/proc/sys`

An alert cordons the node automatically, snapshots it for forensics, and replaces it. Every alert carries the `submissionId` occupying that slot at the time, so the responsible source code is one query away — and it is retained for 14 days alongside the per-test-case execution record.

11.5 Invariant: host execution never leaves a laptop

V0 executes code directly on the host with `child_process` and no isolation, so that the editor and the `/run` path can be built before the sandbox exists. This is a development affordance and a serious vulnerability if it ever ships.

Invariant. The host executor is unreachable outside local development, enforced independently at four levels — a boot guard that throws on `NODE_ENV=production`, a CI check that fails the build if the production bundle imports it, an env schema that rejects the enabling flag in production, and the fact that **V0 is never deployed at all**: staging begins at V1, after the sandbox lands. The package is deleted outright when V1 merges. Implementation in **LLD §6.6**.

---

12 Key Decisions & Tradeoffs

DECISION	CHOSEN	ALTERNATIVE	WHY
Dispatch vs event log	Redis/BullMQ for the **work queue**, Kafka for the **event log**	Kafka for both	Kafka is a log, not a work queue — no per-message ack, no visibility timeout, no priorities, and long-running consumers trigger rebalance storms. A work queue gives us leases, retries and a priority lane for `/run`; Kafka gives us replayable fan-out for the leaderboard, analytics and audit. Using each for what it is good at is the honest answer, and a better interview answer than "we added Kafka". **Adoption trigger — Kafka is not built until it is earned.** V1 ships BullMQ alone, with the worker updating the standings projection directly. Kafka enters in V2 *only if a second independent consumer of judged events actually exists* — analytics, notifications, or an audit trail somebody queries. If the projector is still the only consumer when V2 opens, Kafka is deferred and revisited at V3. Two message systems is a real operational cost, and it has to be paid for by a real second consumer rather than by an architecture diagram.
Isolation	Docker + seccomp	gVisor, Firecracker, `isolate`	Docker with dropped capabilities is enough for a self-hosted, non-nation-state threat model and is by far the fastest to ship. Upgrade path to gVisor documented for the day we host untrusted public contests.
Primary datastore	MongoDB	PostgreSQL	Chosen for MERN continuity and flexible statements. Noted risk: contest scoring is inherently relational and transactional. If scoring correctness bites, submissions and contests move to Postgres while problems stay in Mongo.
Container strategy	Cold `docker run` per submission in V1, warm pool in V3	Warm pool from day one	A ~400 ms cold start is affordable at our rate and is the strictly safer default — a fresh filesystem every time. Optimise when measurements demand it, not before.
Verdict delivery	SSE per submission, WebSocket for standings	Polling everywhere	Polling at contest scale is exactly the read storm the projection was built to avoid.

DECISION	CHOSEN	ALTERNATIVE	WHY
AI grounding	Execution-backed for modes 3 and 4	Pure LLM for all six	A traced run and a measured runtime curve are facts. This is the project's differentiator and it costs little, because the execution plane already exists.
AI coupling	None — separate service, separate failure domain	Mentor embedded in the API service	Embedding is simpler right up until the first provider outage, at which point hung LLM calls occupy API workers and submissions queue behind them. The differentiator must not be able to take down the product. Enforced by lint and a dependency-graph check, verified by a chaos drill — §8.8.

13 Deployment

ENVIRONMENT	SHAPE
Local	`docker-compose up` — api, web, mongo, redis, kafka (redpanda), minio, one worker
Staging	Single EC2, the same compose file, real domain, Let's Encrypt
Production (L4)	ECS or EKS. API, projector and mentor on a general node group. **Judge workers on a dedicated, tainted node group** — untrusted workloads never share a node with the control plane, and that pool is hardened as described in §11.3: no cloud credentials, no cluster secrets, no egress route, ephemeral 24-hour nodes. Autoscaled by KEDA on queue depth, within bounds derived from the measured values in §7. S3 for code and test cases. Blue/green, with the golden-verdict suite as the promotion gate.

14 Testing Strategy

LAYER	WHAT
Unit	Comparators (trailing newline, CRLF, float tolerance), score packing, state machine transitions
Integration	Real Docker. A fork bomb must yield `RE`, an infinite loop `TLE`, a 1 GB allocation `MLE`, and a network call must fail.
Golden verdicts	~50 fixed (problem, code, expected verdict) pairs covering every terminal state. Runs on every deploy; one mismatch blocks promotion. This is the regression net that keeps the judge trustworthy.
Load	k6 contest profile — 2,000 VUs, 8 submissions each, 30% concentrated in the last ten minutes. Pass = p95 verdict under 30 s and zero dropped submissions.
Capacity validation	Re-derive every assumption in §7 from the load test — judge duration, container overhead, slots per node, peak concentration — and rewrite §7 with measured values. **The deliverable is an amended capacity model, not a pass/fail.** Autoscaling bounds are set from the result, not from the estimate.
Chaos	`kill -9` a worker mid-judge (the verdict must still land via redelivery); flush Redis mid-contest (standings must rebuild from Kafka); scale the mentor to zero mid-contest (judging and standings must be completely unaffected — §8.8).

LAYER	WHAT
Security	Attempt filesystem escape, network egress, `/proc` snooping, and reading another submission's scratch dir. All must fail. Separately, verify the containment layer directly: from a shell on a judge node, confirm instance metadata is unreachable, no cloud credential resolves, and no outbound connection to the internet succeeds.

15 Observability

Metrics. `submission_queue_depth` (the autoscaling signal), `judge_duration_seconds` (histogram by language), `verdict_total{verdict}`, `sandbox_oom_total`, `sandbox_timeout_total`, `worker_slots_busy`, `ai_tokens_total{mode}`, `leak_guard_triggered_total`.

Traces. One OpenTelemetry trace per submission spanning API → enqueue → worker → Kafka → projector. `submissionId` is the correlation ID everywhere, logs included.

Alerts. Queue depth above 500 for five minutes · p95 verdict above 60 s · DLQ non-empty · `IE` rate above 1% · fewer than two healthy workers.

16 Risks & Open Questions

RISK	MITIGATION / DECISION NEEDED
Scraping third-party statements — copyright and ToS	Metadata-only by default, official APIs where they exist, our own problems for the flagship set. **Decide before writing the ingestor.**
Docker escape via a kernel CVE	Assumed to happen eventually. Blast radius bounded by §11.3 — credential-less, egress-denied, ephemeral nodes — and detected by syscall monitoring (§11.4). Migration to gVisor or Firecracker is costed and ready if we ever host public untrusted contests.
Capacity assumptions wrong by more than 2×	Autoscaling bounds come from measurements, not from §7's arithmetic; the queue absorbs the error and the p95 SLO is the guardrail that surfaces it. The V4 load test (§14) is scheduled before the first real contest, not after.
Running two message systems is more operational load than the project can carry	Kafka is gated on a second real consumer existing (§12). If it never does, the system stays on BullMQ and loses nothing.
AI cost at scale	Budgets and caching from day one, not retrofitted. Track cost per active user weekly.
Mongo for contest scoring	Accepted for now. Revisit if scoring bugs appear — see §12.
Scope creep across four versions	Each version is independently shippable and demoable. Nothing from V3 starts before V2 is deployed.

17 Milestones

WEEKS	DELIVERABLE	DEMO THAT PROVES IT
1–2	**V0** — auth, problems, editor, `/run` on host	Run “sum of two numbers” from the browser
3–5	**V1** — Docker sandbox, test cases, async `/submit`, verdicts, contests	Submit a wrong answer and get `WA` on test case 7
6–8	**V2** — Kafka, live leaderboard, S3, autoscaling, tracing	Run a 200-user mock contest with live standings
9–11	**V3** — AI Mentor, six modes, execution-grounded	The Dry-Run Simulator finds an off-by-one; the Complexity Auditor calls an $O(n^2)$
12+	**V4** — load tests, chaos, blue/green, runbooks	k6 report: 2,000 VUs, p95 under 30 s

Glossary. `AC` Accepted · `WA` Wrong Answer · `TLE` Time Limit Exceeded · `MLE` Memory Limit Exceeded · `RE` Runtime Error · `CE` Compilation Error · `OLE` Output Limit Exceeded · `IE` Internal Error, our fault rather than the user's · **DLQ** Dead Letter Queue · **CAS** Compare-And-Set · **Checker** the component that decides whether output matches expected.